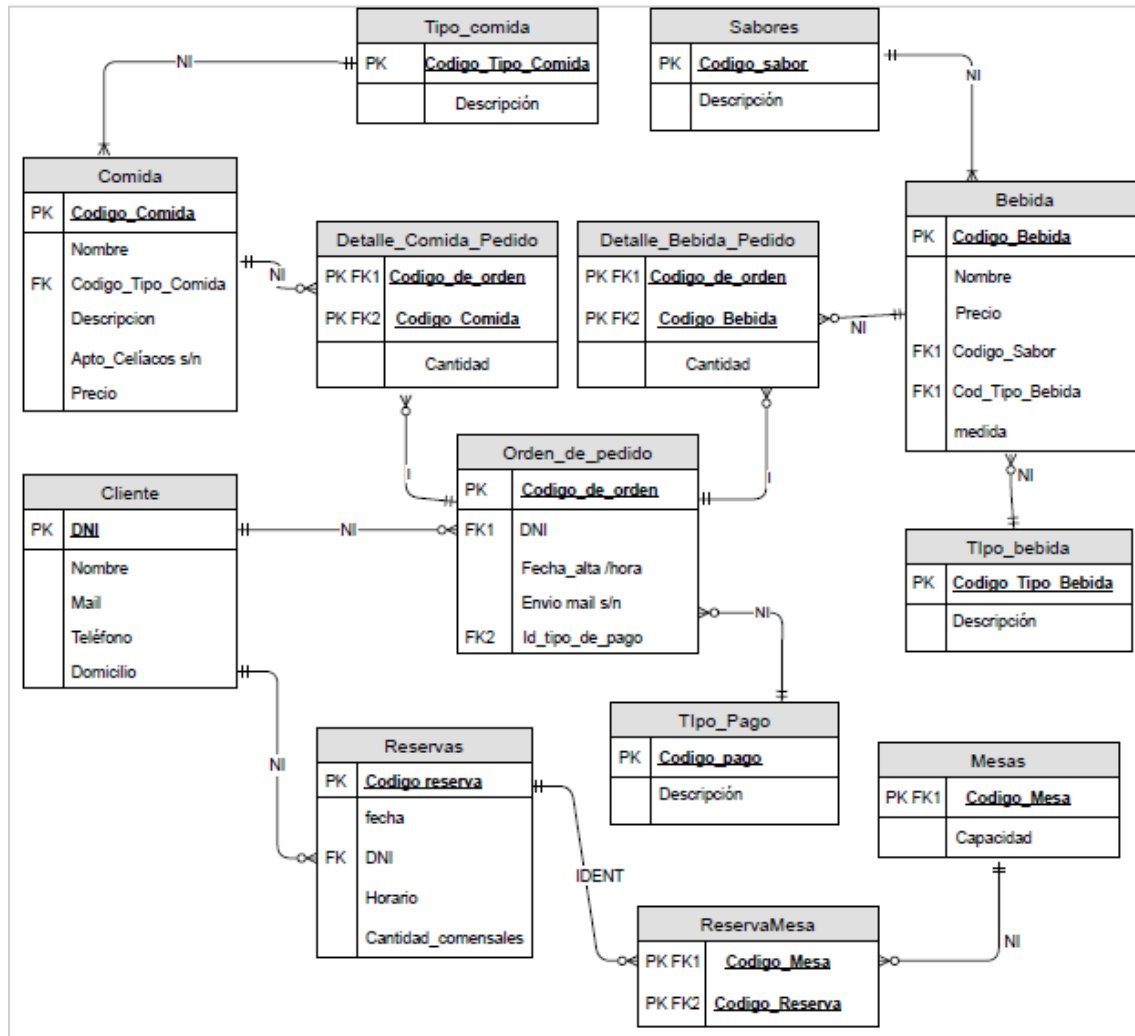


## Guía de ejercicios de parcial – SQL

### Caso del restaurante



- 1- Traer los datos (DNI y Nombre) de los clientes que hicieron ordenes de pedido que incluyeran mas de tres bebidas diferentes
- 2- Mostrar en pantalla los precios maximos, minimos y promedio de todas las comidas. Hacer lo mismo con las bebidas, y presentar los dos resultados a traves de la misma consulta.
- 3- Traer un listado de todos los codigos de ordenes realizados por personas que nunca hayan hecho una reserva en el restaurante. Ordenar los codigos de orden de manera creciente.
- 4- Ordenar correctamente la siguiente consulta

```
SELECT      Codigo_de_orden
FROM        Orden_de_Pedido  A INNER JOIN Cliente B      ON B.DNI = A.DNI
ORDER BY    ASC Codigo_de_orden
WHERE       B.DNI IN NOT
(SELECT DNI  FROM Reservas)
```

- 5- Fettuccini quiere finalmente incorporar el Brunch entre sus tipos de platos disponibles, por lo que servirá algunos platos previamente disponibles en version Brunch. Para esto, se le pide que cargue nuevamente las comidas con los codigos 100, 101 y 102, asignandoles respectivamente los codigos 200, 201 y 202, y cambiando su tipo de comida. Complete los campos faltantes en las siguientes querys

```
INSERT INTO Tipo_Comida VALUES (10, -----)
```

```
INSERT INTO -----
```

```
SELECT '200', Nombre, -----, Descripcion, Apto_Celiacos, ----- * 0.85
```

```
FROM Comida ----- Codigo_Comida = '100'
```

```
INSERT INTO -----
```

```
SELECT '201', Nombre, -----, Descripcion, Apto_Celiacos, ----- * 0.85
```

```
FROM Comida ----- Codigo_Comida = '101'
```

```
INSERT INTO ----- SELECT '202', Nombre, -----, Descripcion,
```

```
Apto_Celiacos, ----- * 0.85 FROM Comida ----- Codigo_Comida = '102'
```

- 6- Indique para qué sirve la siguiente query

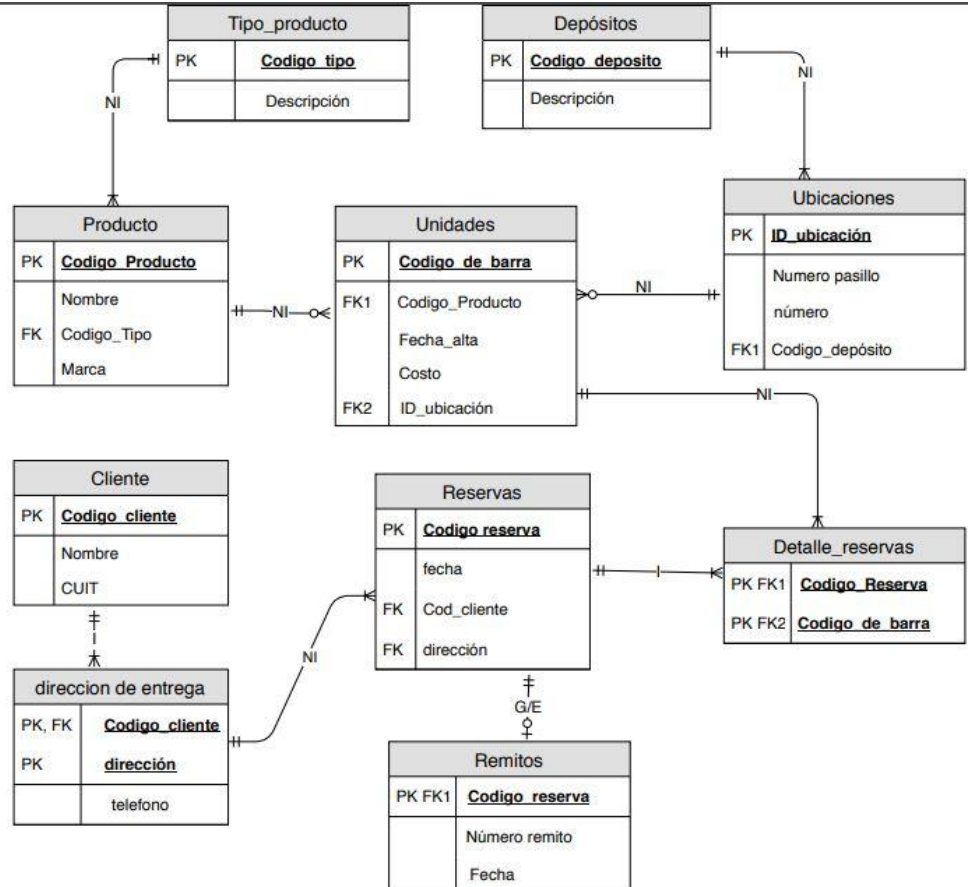
```
SELECT      A.DNI,
            A.Fecha_Alta,
            SUM(Subtotal) as "Cantidad Total",
FROM        Orden_de_pedido  A
INNER JOIN
            (SELECT      B.Codigo_De_Orden, B.Codigo_Comida,
            (B.Cantidad * C.Precio) AS Subtotal
            FROM  Detalle_Comida_Pedido B INNER JOIN Comida C ON
            B.Codigo_Comida = C.Codigo_Comida
            WHERE      B.Codigo_de_Orden = '555') AS D ON
            A.Codigo_de_orden = D.Codigo_De_Orden
INNER JOIN
            (SELECT      E.Codigo_De_Orden, E.Codigo_Bebida,
            (E.Cantidad * F.Precio) AS Subtotal
            FROM  Detalle_Bebida_Pedido E INNER JOIN Comida F ON
            E.Codigo_Bebida = F.Codigo_Bebida
            WHERE      C.Codigo_de_Orden = '555') AS G ON
            A.Codigo_de_orden = G.Codigo_De_Orden
WHERE      A.Codigo_de_orden = '555'
GROUP BY  A.DNI, A.Fecha_Alta
```

7. El cliente Scott Elliott (DNI 12345678) ha sido considerado un cliente no-grato en el restaurant, por lo que no se tomaran mas sus pedidos a domicilio ni reservas de mesa. Cree un trigger que, en el caso de que haya hecho un pedido o una reserva en el sistema, automaticamente se eliminen todos los registros creados para reserva, pedido, y sus datos como cliente.

8. Crear un procedimiento almacenado a cargo de aumentar los precios de las comidas y bebidas del restaurante, usando como parametro de entrada un numero entero (ej. "50" -> 50%). Las actualizaciones de este procedimiento solo deben realizarse sobre las comidas aptas para celiacos, y sobre las bebidas de tipo "Vino"

## Caso TecnoDatos

TecnoDatos es una empresa dedicada a la comercialización de computadoras y notebooks. Debido a la alta demanda por la pandemia, la empresa ha decidido mejorar su sistema de control de stock para poder llevar un control riguroso de las entradas y salidas de mercadería.



- 1) Listar Nombre y CUIT de cada cliente que tenga al menos dos direcciones de entrega
- 2) Se desea saber los detalles de todas las unidades en existencia por marca. Armar un listado mostrando marcas, el costo sumado de todos los productos para esa marca, la cantidad de productos, y el costo promedio.
- 3) Corrija los errores de la siguiente query

```

SELECT          CUIT, C.PrimerReserva
FROM            Cliente C INNER JOIN
              (SELECT      Cod_Cliente, MIN(fecha) AS PrimeraReserva
               FROM          Reservas
               GROUP BY     Cod_Cliente, MIN(fecha))
IN C.Cod_Cliente = Cod_Cliente

```

- 4) Actualizar en un 30% los costos de los productos que se encuentren en los depósitos n° 1 y n° 3

- 5) Determinar cual fue el costo total incurrido por la reserva con el codigo 10005 – Completar los datos faltantes

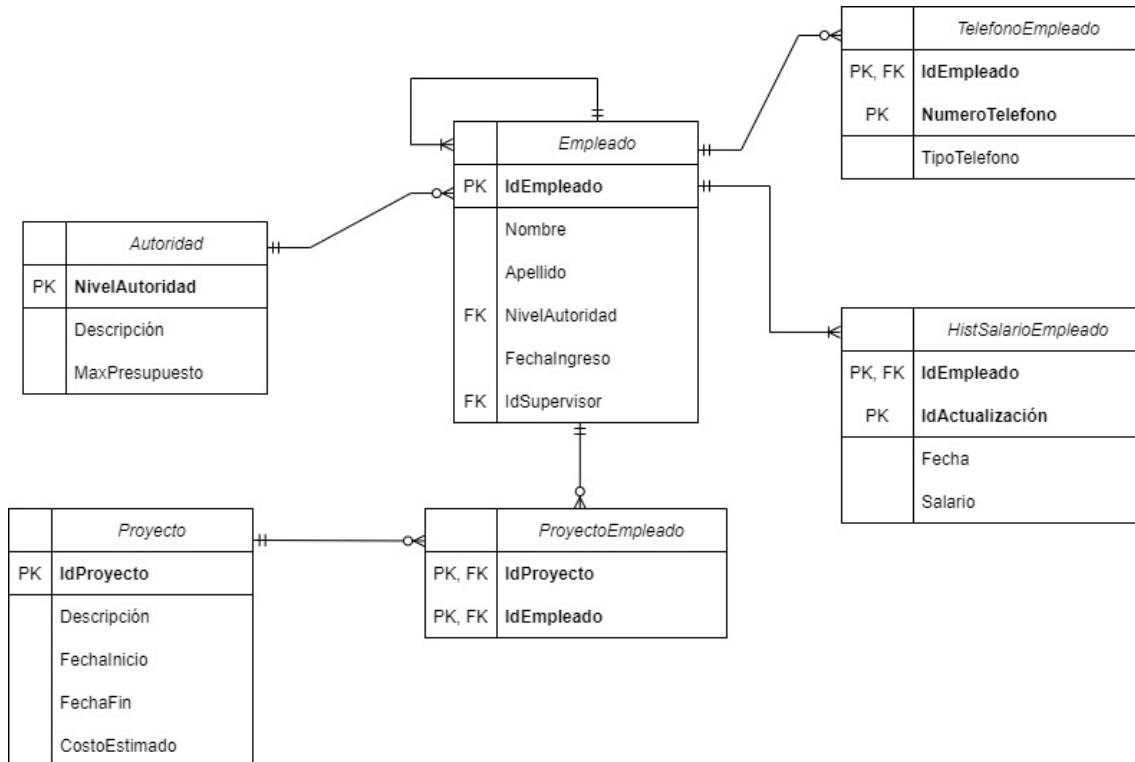
```

SELECT          -----(Costo)
FROM            Reservas R  ----- Detalle_Reservas DR ON
R.Codigo_Reserva = -----
                ----- Unidades U   ON DR.Codigo_de_barra =
U.Codigo_de_barra
----- R.Codigo_Reserva -----

```

- 6) Al finalizar el día, el gerente del depósito, desea contar con un reporte de todas las unidades reservadas pendientes de remitir. El mismo debe incluir el numero de reserva, la fecha de la reserva, el nombre del cliente, el código de barras de cada unidad y el nombre del producto
- 7) Crear un trigger que, al eliminar una reserva remitida, elimine todos los registros de todas las tablas que la involucren. La eliminación debe hacerse en el orden apropiado.
- 8) Crear un procedimiento almacenado al que se le pueda especificar un rango de dos fechas de alta, y que devuelva un listado con todas las unidades disponibles por código de barra, el nombre del producto, y la descripción del tipo de producto. Ordenar de manera creciente por Nombre de Producto.

## Caso de la Gestión de proyectos



- 1- Aumentar todos los presupuestos máximos de los niveles de autoridad 3, 7 y 9 en un 20%
- 2- El costo estimado de un proyecto no debería exceder el presupuesto máximo de ninguno de los empleados participantes. Conseguir los IDs de proyectos y sus Descripciones en donde NO se cumpla esta condición.
- 3- Listar a todos los empleados (por IDEmpleado, Nombre y Apellido) que tengan 3 o mas teléfonos, pero que no dispongan de un tipo de teléfono 'Ofic.'
- 4- Que representa la siguiente query?

```

SELECT      A.Nombre, A.Apellido, B.Nombre, B.Apellido
FROM        Empleado A INNER JOIN Empleado B
           ON A.IdSupervisor = B.IdEmpleado
WHERE       B.IdEmpleado IN (10, 15, 20)
    
```

- 5- Para el empleado nº10, obtener en base a todos los proyectos en los que participa, el costo estimado total acumulado, la menor fecha de inicio, y la mayor fecha de fin. Completar los campos faltantes

```

SELECT      SUM(CostoEstimado), MIN(FechaInicio), -----
FROM        Empleado A ----- ProyectoEmpleado B
           ON A.IdEmpleado = -----
INNER JOIN Proyecto C ON B.IdProyecto = C.IdProyecto
WHERE       A.IdEmpleado = -----
----- A.IdEmpleado
    
```

- 6- Listar para los últimos 5 proyectos iniciados (pero NO terminados) sus descripciones, fecha de inicio, costo estimado y participantes (nombre, apellido, nivel de autoridad y salario actual)
- 7- Crear un trigger que, en el caso de que se elimine un empleado, y este sea supervisor, actualizar el valor de IdSupervisor para sus empleados a cargo a 0 (temporalmente sin supervisor)
- 8- Armar un procedimiento almacenado a cargo de actualizar el salario de un empleado. Usar como parámetros IdEmpleado, y un numero entero para porcentaje. La “actualización” debe INSERTAR un nuevo registro

